ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN – ĐIỆN TỬ

ĐỒ ÁN TỐT NGHIỆP

Hệ thống Smart Building với Zigbee và BLE

Lê Đức Anh – 20224403
anh.ld224403@sis.hust.edu.vn

Chu Thiên Phú – 20224450
phu.ct224450@sis.hust.edu.vn

Ngành: Kỹ thuật Điện tử – Viễn thông
Chuyên ngành: Hệ thống nhúng thông minh và IoT

Giáo viên hướng dẫn: Phạm Thị Nhẫn

Chữ ký của GVHD

KHOA: *(Điền tên khoa)*

Hà Nội, 01/2026

LỜI CẢM ƠN

Mặc dù đã cố gắng hoàn thiện đồ án một cách tốt nhất, song do kiến thức và kinh nghiệm còn hạn chế nên không tránh khỏi những thiếu sót. Nhóm rất mong nhận được những ý kiến đóng góp quý báu từ các thầy cô để đồ án được hoàn thiện hơn.

Hà Nội, tháng <...> năm <...>

Nhóm sinh viên thực hiện

MỤC LỤC

DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT **4**

DANH SÁCH HÌNH VẼ **6**

DANH SÁCH BẢNG **7**

PHÂN CHIA CÔNG VIỆC **8**

MỞ ĐẦU **9**
Bối cảnh và vấn đề 9
Mục tiêu và phạm vi 9
Deliverables 9
Cấu trúc báo cáo 10

1 TỔNG QUAN HỆ THỐNG VÀ CƠ SỞ LÝ THUYẾT **11**
1.1 Kiến trúc hệ thống 11
1.1.1 Zigbee Network (2.4 GHz) 11
1.1.2 Coordinator / NCP (Network Co-Processor) 11
1.1.3 Giao tiếp Host – NCP (EZSP/ASH qua UART) 11
1.1.4 Gateway Host (Z3Gateway trên Linux) 12
1.1.5 Cloud Backend 12
1.1.6 Flutter Mobile App 12
1.1.7 Use Cases theo vai trò 12
1.2 Giao thức Zigbee 13
1.2.1 Tổng quan 13
1.2.2 Vai trò thiết bị 13
1.2.3 Topology mạng 13
1.2.4 Kiến trúc Protocol Stack 14
1.2.5 Địa chỉ và Bảo mật 15
1.2.6 Quá trình hình thành và gia nhập mạng 15
1.3 Zigbee Cluster Library (ZCL) 16
1.3.1 Cấu trúc ZCL 16
1.3.2 Binding (Liên kết) 17
1.3.3 Group (Nhóm) 17
1.4 Giao thức MQTT 17
1.4.1 Tổng quan 17
1.4.2 Thành phần chính 17
1.4.3 Quality of Service (QoS) 17
1.4.4 Retained Messages và Last Will 18
1.4.5 Topics và Wildcards 18
1.5 Giao tiếp UART 18
1.5.1 Tổng quan 18
1.5.2 Frame Format 19
1.5.3 Ứng dụng UART trong dự án: EZSP/ASH 19

1.6 Phần cứng và Kit sử dụng . . . 19
1.6.1 EFR32MG12 2.4 GHz 10 dBm Wireless Starter Kit . . . 19
1.6.2 Cấu hình phần cứng theo thiết bị . . . 20
1.6.3 Toolchain và SDK . . . 20
1.7 Luồng dữ liệu (Data Flow) . . . 20
1.7.1 Luồng Uplink (State/Event Report) . . . 20
1.7.2 Luồng Downlink (Command) . . . 21
1.7.3 Luồng Local Control . . . 21
1.8 MQTT Contract . . . 21
1.8.1 Namespace và Topic Structure . . . 21
1.8.2 Message Envelope . . . 22
1.8.3 Retain và QoS Policy . . . 22

2 THIẾT KẾ HỆ THỐNG 23
2.1 Yêu cầu chức năng và phi chức năng . . . 23
2.1.1 Yêu cầu chức năng . . . 23
2.1.2 Yêu cầu phi chức năng . . . 23
2.2 Network và Device Roles (Zigbee) . . . 23
2.3 Data và Message Design . . . 24
2.3.1 MQTT Message Channels . . . 24
2.3.2 State Message Design . . . 24
2.3.3 Command Message Design . . . 24
2.3.4 Versioning Rule . . . 25
2.4 Logic Flow và State Machine . . . 25
2.4.1 Light Device State Machine . . . 25
2.4.2 Command Lifecycle . . . 25
2.4.3 Gateway Runtime Flow . . . 26
2.5 Cloud Backend Architecture . . . 26
2.5.1 Kiến trúc tổng quan . . . 26
2.5.2 Database Design . . . 26
2.5.3 REST API Endpoints . . . 27
2.5.4 MQTT – Database Integration . . . 27
2.6 Trade-offs và Quyết định thiết kế . . . 27

3 TRIỂN KHAI HỆ THỐNG 28
3.1 Triển khai Firmware . . . 28
3.1.1 Cấu trúc Firmware Coordinator . . . 28
3.1.2 Firmware Light Device . . . 28
3.1.3 Firmware Switch Device . . . 28
3.1.4 Firmware Occupancy Sensor . . . 28
3.2 Triển khai Gateway Service . . . 29
3.2.1 Thiết lập môi trường . . . 29
3.2.2 Kiến trúc đa luồng . . . 29
3.2.3 Giao tiếp Host–NCP: EZSP/ASH và MQTT trực tiếp . . . 29
3.2.4 Cơ chế Correlation ID . . . 29
3.2.5 Chính sách Timeout và Retry . . . 29
3.2.6 Rules Engine . . . 29
3.2.7 Admin API . . . 29
3.3 Triển khai Cloud (FastAPI + PostgreSQL + EC2) . . . 30

3.3.1 Thiết lập Cloud Stack 30
3.3.2 Device State Sync 30
3.3.3 Event History và Logging 30
3.3.4 Vấn đề thực tế khi deploy EC2 30
3.4 Triển khai Flutter Mobile App 31
3.5 Luồng End-to-end Command/State 31
3.5.1 Client → Cloud → Gateway → Light 31
3.5.2 Local Automation: Occupancy → Light 31
3.6 Hướng phát triển OTA 31

4 KẾT QUẢ VÀ ĐÁNH GIÁ 32
4.1 Điều kiện chạy thử (Test Setup) 32
4.2 Kết quả End-to-end 32
4.3 Kết quả từng thành phần 33
4.3.1 Zigbee Network 33
4.3.2 Gateway Service 33
4.3.3 Cloud Firebase 33
4.3.4 Flutter Mobile App 33
4.4 Đánh giá hiệu năng 33

5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 34
5.1 Kết luận 34
5.2 Hạn chế 34
5.3 Hướng phát triển 35

TÀI LIỆU THAM KHẢO 36

DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT

Viết tắt	Ý nghĩa
ACK	Acknowledgement
AES	Advanced Encryption Standard
API	Application Programming Interface
APS	Application Support Sub-layer
BLE	Bluetooth Low Energy
CID	Correlation ID
CLI	Command Line Interface
CMD	Command
CMSIS	Cortex Microcontroller Software Interface Standard
CRLF	Carriage Return Line Feed
CSA	Connectivity Standards Alliance
CSMA/CA	Carrier Sense Multiple Access with Collision Avoidance
DMA	Direct Memory Access
EZSP	EmberZNet Serial Protocol
FFD	Full Function Device
GBL	Gecko Bootloader
GPIO	General Purpose Input/Output
GSDK	Gecko SDK
GUI	Graphical User Interface
HTTP	Hypertext Transfer Protocol
I2C	Inter-Integrated Circuit
IDE	Integrated Development Environment
IoT	Internet of Things
JSON	JavaScript Object Notation
LCD	Liquid Crystal Display
LED	Light Emitting Diode
LQI	Link Quality Indicator
LWT	Last Will and Testament (MQTT)
MAC	Media Access Control
MCU	Microcontroller Unit
MQTT	Message Queuing Telemetry Transport
NCP	Network Co-Processor
NVM3	Non-Volatile Memory 3rd Generation
NWK	Network Layer
OTA	Over-The-Air (firmware update)
PAN	Personal Area Network
PHY	Physical Layer
PSA	Platform Security Architecture
PTI	Packet Trace Interface
QoS	Quality of Service
RAIL	Radio Abstraction Interface Layer
REST	Representational State Transfer
RF	Radio Frequency
RFD	Reduced Function Device
RPi	Raspberry Pi

Viết tắt	Ý nghĩa
RTOS	Real-Time Operating System
SDK	Software Development Kit
SED	Sleepy End Device
SoC	System on Chip
SPI	Serial Peripheral Interface
SSv5	Simplicity Studio version 5
TCP/IP	Transmission Control Protocol / Internet Protocol
TLS	Transport Layer Security
UART	Universal Asynchronous Receiver-Transmitter
USB	Universal Serial Bus
VCOM	Virtual COM Port
WSTK	Wireless Starter Kit
ZC	Zigbee Coordinator
ZCL	Zigbee Cluster Library
ZDO	Zigbee Device Object
ZED	Zigbee End Device
ZR	Zigbee Router

DANH SÁCH HÌNH VẼ

1.1 Kiến trúc tổng quan hệ thống Smart Home Platform 11
1.2 Star topology – Coordinator ở trung tâm 13
1.3 Mesh topology – Định tuyến qua nhiều node 14
1.4 Tree topology – Cấu trúc phân cấp 14
1.5 Kiến trúc Zigbee Protocol Stack 15
1.6 Mô hình Client/Server trong ZCL 16
1.7 Kiến trúc Publish/Subscribe của MQTT 17
1.8 Kết nối UART giữa hai thiết bị 19
1.9 EFR32MG12 2.4 GHz 10 dBm Wireless Starter Kit 19
1.10 Luồng dữ liệu Uplink 20
1.11 Luồng điều khiển Downlink 21

DANH SÁCH BẢNG

2 Bảng phân chia công việc 8

1.1 Các module chính trong Z3Gateway 12
1.2 Các cluster chính sử dụng trong dự án 16
1.3 Các mức QoS trong MQTT 18
1.4 Toolchain và SDK version 20
1.5 Các nhóm MQTT topic chính 21

2.1 Ánh xạ device type và cluster chính 23
2.2 Thiết kế các kênh MQTT 24
2.3 Các bảng database chính 26
2.4 REST API endpoints chính 27

3.1 Biến môi trường Cloud (prefix `SB_`) 30

PHÂN CHIA CÔNG VIỆC

STT	Công việc	Sinh viên thực hiện
Firmware – Zigbee Devices		
1	<Firmware Coordinator: tạo mạng, Trust Center, ZCL, UART bridge>	<SV>
2	<Firmware Light Device: On/Off Server, LED indicator, state report>	<SV>
3	<Firmware Switch Device: On/Off Client, binding, button control>	<SV>
4	<Firmware Occupancy Sensor: occupancy detection, ZCL report>	<SV>
Gateway Service (Ubuntu / Raspberry Pi)		
5	<Gateway UART module: kết nối COM, đọc line, reconnect>	<SV>
6	<Gateway MQTT bridge: subscribe cmd, publish state/telemetry/ack>	<SV>
7	<Gateway rules engine, ACK routing, retry/timeout>	<SV>
8	<Gateway Admin API (/health, /status, /config, /rules)>	<SV>
Cloud – Firebase		
9	<Firebase setup: Realtime DB / Firestore schema, Cloud Functions>	<SV>
10	<MQTT – Firebase bridge: device state sync, command relay>	<SV>
11	<Event history, logging, device management trên cloud>	<SV>
Mobile App – Flutter		
12	<Flutter app: device list, light on/off control UI>	<SV>
13	<Flutter app: occupancy state display, realtime update>	<SV>
14	<Flutter app: event history, notification>	<SV>
Tổng hợp & Tài liệu		
15	<Tổng hợp và hoàn thiện báo cáo>	<SV 1, SV 2>

Bảng 2: Bảng phân chia công việc

MỞ ĐẦU

1. Bối cảnh và vấn đề

Trong bối cảnh smart home (nhà thông minh) và smart building (tòa nhà thông minh) ngày càng phổ biến, nhu cầu xây dựng hệ thống IoT hoàn chỉnh từ tầng thiết bị đến ứng dụng di động trở thành một bài toán có giá trị cả về mặt kỹ thuật lẫn thực tiễn. Các giải pháp hiện có trên thị trường thường là hệ sinh thái đóng, khó tùy biến và mở rộng. Việc nghiên cứu và xây dựng một nền tảng mở, dựa trên các chuẩn giao tiếp phổ biến như Zigbee và MQTT, cho phép kiểm soát toàn bộ luồng dữ liệu từ thiết bị đến người dùng.

Zigbee được lựa chọn cho tầng local network bởi giao thức này phù hợp với mô hình nhiều thiết bị nhỏ gọn, tiết kiệm năng lượng, hỗ trợ mesh network giúp tăng phạm vi phủ sóng và độ tin cậy. Zigbee đã được ứng dụng rộng rãi trong smart home với các thiết bị chiếu sáng, công tắc, cảm biến hiện diện (occupancy sensor) và nhiều loại thiết bị khác.

2. Mục tiêu và phạm vi

Đề tài hướng tới xây dựng một nền tảng smart home mini-platform dựa trên kiến trúc IoT nhiều lớp, bao gồm:

- **Local network:** Mạng Zigbee kết nối các thiết bị: đèn thông minh (light device), công tắc (switch device), cảm biến hiện diện (occupancy sensor), được điều phối bởi Zigbee Coordinator.
- **Gateway:** Chạy trên Linux (Ubuntu, hướng migrate sang Raspberry Pi), đóng vai trò bridge giữa mạng Zigbee và cloud thông qua UART và MQTT.
- **Cloud:** Sử dụng Firebase cho backend, quản lý device state, command dispatch, event history.
- **Mobile App:** Ứng dụng Flutter hiển thị trạng thái thiết bị, điều khiển bật/tắt đèn, xem occupancy state, nhận cập nhật realtime.

Phạm vi triển khai tập trung vào kiểm chứng luồng dữ liệu end-to-end: App gửi command → Cloud nhận → Gateway dịch sang Zigbee command → Light Device thực thi → State report quay lại Cloud và App.

3. Deliverables

1. Firmware cho 4 node Zigbee: Coordinator, Light Device, Switch Device, Occupancy Sensor.
2. Gateway service chạy trên Ubuntu (Python), bridge UART ↔ MQTT.
3. Cloud backend trên Firebase (Realtime DB / Firestore, Cloud Functions).
4. Ứng dụng di động Flutter với chức năng điều khiển đèn và giám sát occupancy.
5. Tài liệu kỹ thuật: đặc tả giao thức, MQTT topics, kiến trúc hệ thống.

4. Cấu trúc báo cáo

Báo cáo được tổ chức thành 5 chương:

- **Chương 1 – Tổng quan hệ thống và cơ sở lý thuyết:** Trình bày kiến trúc hệ thống, các thành phần, phần cứng sử dụng, và nền tảng lý thuyết Zigbee, ZCL, MQTT, UART.

CHƯƠNG 1
TỔNG QUAN HỆ THỐNG VÀ CƠ SỞ LÝ THUYẾT

Chương này trình bày kiến trúc tổng thể của hệ thống Smart Home Platform theo mô hình phân tầng, bao gồm mạng Zigbee cục bộ, Coordinator/NCP, Z3Gateway host trên Linux, broker MQTT trung gian và Cloud Backend trên AWS EC2. Tiếp theo là cơ sở lý thuyết về giao thức Zigbee, Zigbee Cluster Library, MQTT, giao tiếp UART, phần cứng và kit phát triển, luồng dữ liệu end-to-end và đặc tả MQTT Contract.

1.1 Kiến trúc hệ thống

Hệ thống được thiết kế theo mô hình phân tầng gồm bốn lớp chính: mạng Zigbee cục bộ hoạt động ở băng tần 2.4 GHz, host gateway chạy Linux, broker MQTT trung gian và cloud backend trên AWS EC2. Quyết định kiến trúc quan trọng được đưa ra vào tháng 4/2026 là loại bỏ hoàn toàn cầu nối Python MQTT↔IPC; ứng dụng Z3Gateway viết bằng C nay giao tiếp MQTT trực tiếp, không còn Unix socket hay NDJSON IPC.

Hình 1.1: Kiến trúc tổng quan hệ thống Smart Home Platform

1.1.1 Zigbee Network (2.4 GHz)

Mạng Zigbee trong dự án hoạt động trên băng tần ISM 2.4 GHz theo chuẩn IEEE 802.15.4, sử dụng mô hình Centralized Trust Center với một Coordinator duy nhất. Các thiết bị đầu cuối gồm Z3Light, Z3Switch và Occupancy Sensor gia nhập mạng thông qua cơ chế permit-join do gateway điều khiển. Module `net_mgr.c/h` trong Z3Gateway đảm nhiệm việc form network và mở cửa sổ permit-join.

1.1.2 Coordinator / NCP (Network Co-Processor)

Coordinator được triển khai trên chip Silicon Labs EFR32MG12P332F1024GL125 chạy firmware NCP (Network Co-Processor). NCP không chạy logic ứng dụng mà chỉ đảm nhiệm lớp radio và lớp mạng Zigbee; mọi quyết định ứng dụng được đẩy lên host. Firmware NCP được build riêng trong thư mục `end_devices/NCP`.

Mô hình NCP khác biệt cơ bản so với mô hình SoC: trong SoC, ứng dụng và Zigbee stack chạy chung trên một MCU, phù hợp cho thiết bị đơn giản như đèn hay công tắc; còn NCP tách Zigbee stack ra chip radio riêng, để host có tài nguyên dồi dào cho MQTT, database và tích hợp cloud. Dự án chọn NCP cho Coordinator/gateway và SoC cho các end device.

1.1.3 Giao tiếp Host – NCP (EZSP/ASH qua UART)

Host Z3Gateway trên Linux kết nối với NCP qua UART sử dụng giao thức EZSP (EmberZNet Serial Protocol) chạy trên lớp ASH (Asynchronous Serial Host). Giao thức này thuộc sở

hữu của Silicon Labs và được Z3Gateway xử lý nguyên bản, dự án không tái triển khai framing UART ở lớp này. Cổng UART được cấu hình trong `app_config.h` hoặc biến môi trường.

EZSP cho phép host gửi lệnh điều khiển Zigbee stack trên NCP, chẳng hạn form network, send ZCL command hay read attribute, đồng thời nhận callback và event từ NCP như device joined, attribute report hay message received. ASH cung cấp cơ chế ACK, retry, CRC và flow control để đảm bảo truyền tin cậy trên UART.

1.1.4 Gateway Host (Z3Gateway trên Linux)

Gateway là ứng dụng C dựa trên Z3GatewayHost của Silicon Labs, build qua `Z3Gateway.Makefile` từ Simplicity Studio. Các module ứng dụng nằm trong `gateway/Z3Gateway/Z3GatewayHost/app/`.

Bảng 1.1: Các module chính trong Z3Gateway

Module	**Vai trò**
`app_mqtt.c/h`	Client MQTT: publish reported/event/reply, subscribe desired/command
`cmd_handler.c/h`	Parse MQTT envelope, dispatch command
`device_registry.c/h`	Bảng thiết bị trong RAM (eui64, nwk_addr, type)
`device_dispatch.c/h`	Dispatch lệnh theo loại thiết bị
`light_ctrl.c/h`	Điều khiển cluster On/Off (0x0006)
`switch_logic.c/h`	Logic toggle/state của switch
`telemetry_rx.c/h`	ZCL attribute report → MQTT reported
`device_monitor.c/h`	Theo dõi presence / last-seen
`net_mgr.c/h`	Form network, permit-join
`rule_engine.c/h, sb_command.c/h`	Rule evaluation, command helpers

Kiến trúc runtime là single-process: MQTT client được tích hợp trực tiếp trong cùng tiến trình C với EZSP host logic. Gateway đóng vai trò bridge duy nhất giữa thế giới IP và mạng Zigbee RF, chuẩn hóa dữ liệu từ ZCL sang định dạng JSON qua MQTT.

1.1.5 Cloud Backend

Cloud backend là một REST API viết bằng FastAPI 0.115 (Python 3.12) kết hợp client MQTT và worker timeout chạy nền. Dữ liệu được lưu trong PostgreSQL qua SQLAlchemy 2.0 async cùng asyncpg. Môi trường production được đóng gói trong `deploy/docker-compose.prod.yml` gồm ba container: `sb-postgres`, `sb-mosquitto` và `sb-cloud-api` chạy trên EC2.

1.1.6 Flutter Mobile App

Tính năng này chưa được triển khai trong phạm vi dự án. Thư mục `mobile_app/` hiện chỉ là placeholder, được hoãn sau bản v1.

1.1.7 Use Cases theo vai trò

Người dùng cuối gọi REST API (`POST /api/devices/{id}/command`) để điều khiển đèn. Hệ thống cloud publish lệnh MQTT tới gateway, gateway dispatch xuống cluster On/Off của thiết bị Zigbee, sau đó nhận phản hồi và cập nhật trạng thái ngược về database. Quản trị viên có thể seed dữ liệu mẫu qua `python -m cloud.app.seed` hoặc `deploy/seed-remote.ps1`.

1.2 Giao thức Zigbee

1.2.1 Tổng quan

Zigbee là giao thức mạng không dây năng lượng thấp dựa trên IEEE 802.15.4, tối ưu cho mesh network mật độ cao trong nhà thông minh. Dự án sử dụng Zigbee 3.0 (Z3) thông qua ngăn xếp EmberZNet của Silicon Labs. Giao thức hoạt động chủ yếu trên dải tần 2.4 GHz với tốc độ 250 kbps, hỗ trợ bảo mật AES-128 bit với Trust Center và phù hợp cho các ứng dụng IoT, tự động hóa nhà và công nghiệp.

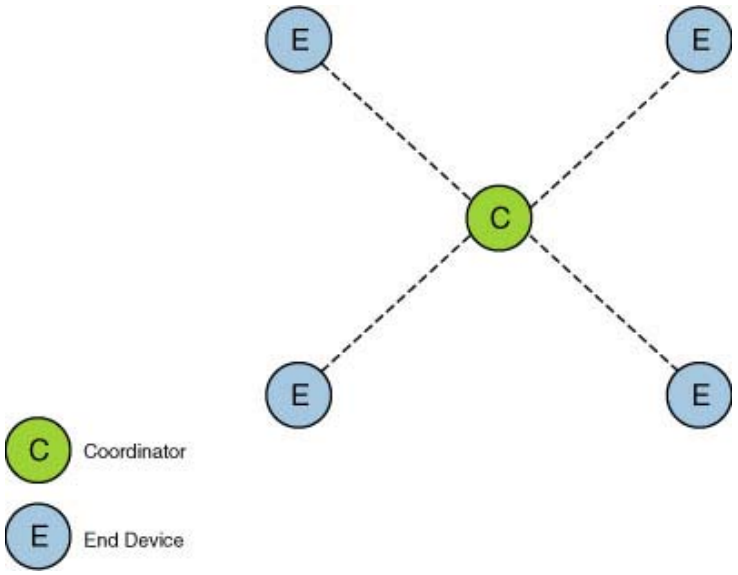
1.2.2 Vai trò thiết bị

Mạng có ba vai trò thiết bị. Coordinator là thiết bị duy nhất chạy firmware NCP trên EFR32MG12, chịu trách nhiệm khởi tạo và quản lý mạng, đóng vai trò Trust Center với địa chỉ luôn là `0x0000`. Router là các thiết bị nguồn lưới như Z3Light và Z3Switch, có khả năng relay gói tin và mở rộng phạm vi mạng, luôn bật và không sleep. End Device là thiết bị đầu cuối tiết kiệm năng lượng như Occupancy Sensor, chỉ giao tiếp qua thiết bị cha và có thể sleep.

1.2.3 Topology mạng

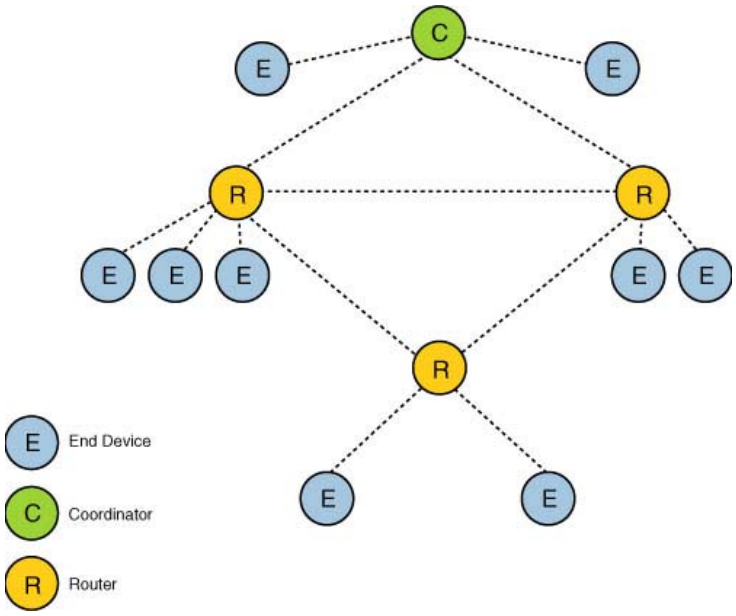
Zigbee hỗ trợ nhiều mô hình mạng khác nhau tùy theo yêu cầu ứng dụng.

Mạng hình sao (Star Topology) là mô hình đơn giản nhất với Coordinator ở trung tâm và các End Device kết nối trực tiếp. Ưu điểm là dễ triển khai, End Device có thể sleep tiết kiệm pin. Nhược điểm là phạm vi hạn chế và Coordinator là single point of failure.



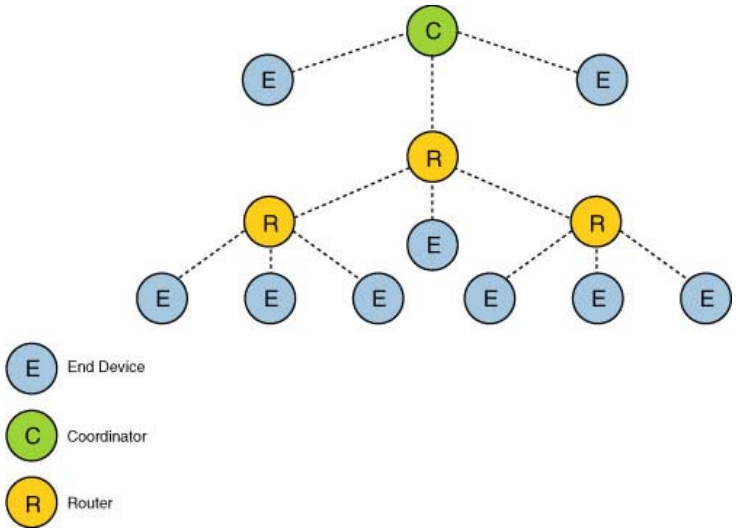
Hình 1.2: Star topology – Coordinator ở trung tâm

Mạng lưới (Mesh Topology) là mô hình mạnh mẽ nhất với Coordinator và nhiều Router tạo mạng lưới. Router có thể chuyển tiếp dữ liệu multi-hop, tự tìm đường thay thế khi có node lỗi, phù hợp cho hệ thống lớn cần độ tin cậy cao.



Hình 1.3: Mesh topology – Định tuyến qua nhiều node

Mạng hình cây (Tree Topology) có cấu trúc phân cấp với Coordinator là gốc, Router làm nhánh và End Device là lá. Mô hình này kết hợp ưu điểm của Star (đơn giản) và Mesh (mở rộng), thường dùng trong các ứng dụng cần phân cấp quản lý rõ ràng.

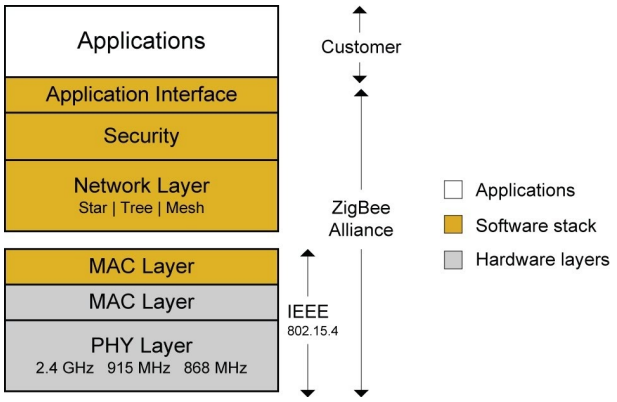


Hình 1.4: Tree topology – Cấu trúc phân cấp

Mô hình triển khai thực tế trong dự án là mesh tree: Coordinator ở gốc, Router mở rộng vùng phủ, End Device gắn vào Router gần nhất. Module `device_registry.c` lưu mapping eui64 ↔ nwk_addr ↔ device_type để gateway định tuyến lệnh xuống đúng thiết bị.

1.2.4 Kiến trúc Protocol Stack

Kiến trúc Zigbee Protocol Stack được xây dựng dựa trên mô hình phân tầng, trong đó mỗi lớp cung cấp dịch vụ cụ thể cho lớp bên trên.



Hình 1.5: Kiến trúc Zigbee Protocol Stack

Stack gồm PHY/MAC (IEEE 802.15.4) ở tầng thấp nhất, trong đó PHY xử lý sóng vô tuyến 2.4 GHz và điều chế tín hiệu, MAC quản lý truy cập kênh theo CSMA-CA và kiểm soát khung tin cậy. Tiếp theo là lớp NWK (Zigbee Network Layer) do Zigbee định nghĩa, chịu trách nhiệm định tuyến AODV qua mạng mesh, quản lý địa chỉ mạng 16-bit và xử lý bảo mật mạng bằng Network Key. Lớp APS (Application Support Sublayer) đóng vai trò cầu nối giữa NWK và ứng dụng, xử lý binding, phân mảnh tin nhắn và lọc gói tin trùng lặp. Trên cùng là ZCL (Zigbee Cluster Library), lớp ứng dụng định nghĩa mọi hành vi thiết bị. Dự án chỉ can thiệp ở lớp ZCL và APS thông qua các module trong `Z3GatewayHost/app/`.

Bên cạnh các lớp trên là ZDO (Zigbee Device Object) nằm ở Endpoint 0, chịu trách nhiệm khởi tạo thiết bị, khám phá mạng và quản lý yêu cầu binding. Security Service Provider (SSP) cung cấp dịch vụ mã hóa và xác thực cho cả lớp NWK lẫn APS.

1.2.5 Địa chỉ và Bảo mật

Mỗi thiết bị có eui64 (64-bit MAC) là định danh vĩnh viễn, được dùng làm khóa bền trong payload MQTT. Nwk_addr (16-bit runtime) chỉ dùng cho debug vì nó thay đổi mỗi lần thiết bị rejoin mạng, tuyệt đối không được sử dụng làm primary key. Cloud lưu `device_id` riêng làm khóa logic trong PostgreSQL, tránh gắn chặt vào phần cứng và không leak thông tin MAC ra ngoài.

Về bảo mật, mạng sử dụng Network Key 128-bit chia sẻ cho tất cả thiết bị trong mạng, Link Key riêng giữa từng cặp thiết bị, và Trust Center do Coordinator quản lý để xác thực và phân phối khóa.

1.2.6 Quá trình hình thành và gia nhập mạng

Gateway gọi `net_mgr.c` để form network bằng cách quét năng lượng (Energy Scan) chọn kênh vô tuyến ít nhiễu nhất và chọn một PAN ID duy nhất, sau đó mở permit-join trong khoảng thời gian cấu hình.

Quá trình End Device gia nhập mạng diễn ra theo bốn giai đoạn. Giai đoạn khám phá mạng (Network Discovery): End Device gửi Beacon Request, Coordinator trả lời bằng Beacon chứa cờ Association Permit = TRUE, End Device chọn Coordinator làm thiết bị cha. Giai đoạn kết nối MAC (MAC Association): End Device gửi Association Request chứa EUI-64 và Capability Info, Coordinator cấp phát địa chỉ mạng 16-bit qua Association Response. Giai đoạn xác thực và trao đổi khóa: Coordinator đóng vai trò Trust Center, gửi Transport Key chứa Network Key đã mã hóa bằng Link Key ban đầu (`ZigbeeAlliance09` hoặc Install Code), End Device giải mã và đạt trạng thái "Joined and Authenticated". Giai đoạn thông báo và cập nhật bảo mật:

End Device phát quảng bá Device_annce cho toàn mạng, sau đó yêu cầu đổi Link Key mới (Trust Center Link Key) để thay thế khóa mặc định.

Khi join thành công, thiết bị được đưa vào bảng `device_registry` và `telemetry_rx.c` bắt đầu nhận các attribute report.

1.3 Zigbee Cluster Library (ZCL)

ZCL là thư viện chuẩn hóa định nghĩa mọi chức năng mà một thiết bị Zigbee có thể thực hiện, đóng vai trò như "cuốn từ điển ngôn ngữ" chung đảm bảo tương tác giữa các sản phẩm của nhiều nhà sản xuất.

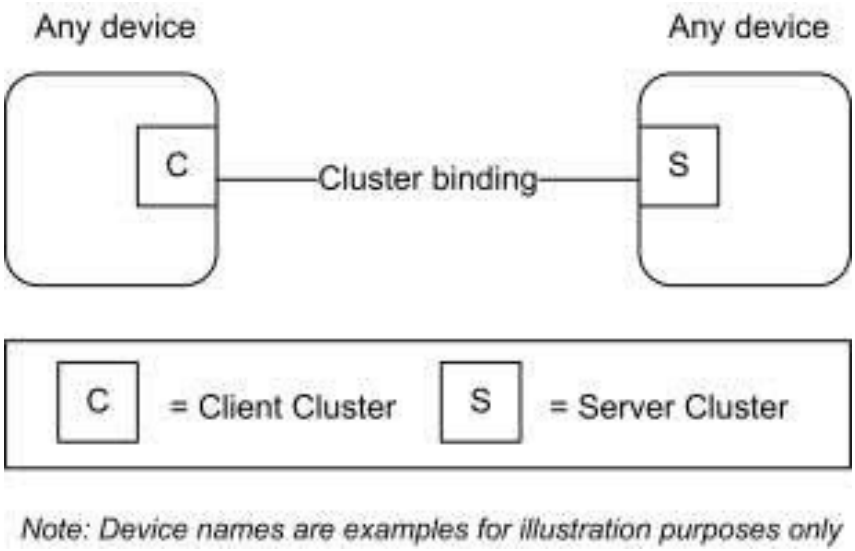
1.3.1 Cấu trúc ZCL

ZCL chia chức năng thành các cluster định danh bằng ID 16-bit. Mỗi cluster đại diện cho một chức năng cụ thể, chứa các attribute lưu trữ trạng thái và các command thực hiện hành động. Mỗi cluster chia làm hai phía: Server là nơi chứa attribute thực tế và thực thi lệnh (ví dụ bóng đèn chứa On/Off Server), Client là nơi gửi lệnh điều khiển (ví dụ công tắc chứa On/Off Client).

Bảng 1.2: Các cluster chính sử dụng trong dự án

Cluster ID	**Tên**	**Dùng cho**
0x0006	On/Off	Light (bật/tắt)
(client)	On/Off client + binding	Switch
0x0406	Occupancy Sensing	Occupancy Sensor

Cluster On/Off (0x0006) được `light_ctrl.c/h` gọi để bật/tắt đèn. Attribute report của cluster này được `telemetry_rx.c/h` bắt và chuyển thành message MQTT reported.



Hình 1.6: Mô hình Client/Server trong ZCL

1.3.2 Binding (Liên kết)

Binding là cơ chế APS cho phép thiết bị nguồn (switch) gửi lệnh trực tiếp tới thiết bị đích (light) mà không cần qua gateway. Khi ứng dụng gửi lệnh qua cluster đã được binding, Zigbee Stack tự tra Bảng Binding để điền địa chỉ đích vào gói tin. Đây là nền tảng cho luồng local control (xem Mục 1.7.3), cho phép Switch điều khiển Light trực tiếp trong mạng mesh Zigbee với độ trễ rất thấp.

1.3.3 Group (Nhóm)

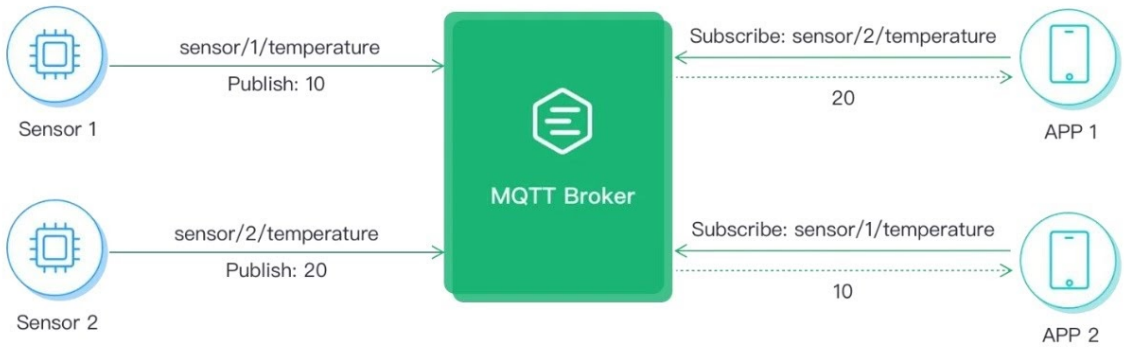
Group cho phép nhiều thiết bị cùng nhận một lệnh broadcast cluster thông qua một Group ID 16-bit duy nhất, thay vì gửi nhiều lệnh Unicast riêng lẻ. Trong phạm vi hiện tại, chi tiết triển khai group chưa nằm trong code gateway, chưa có helper riêng trong `Z3GatewayHost/app/`.

1.4 Giao thức MQTT

1.4.1 Tổng quan

MQTT (Message Queuing Telemetry Transport) là giao thức nhắn tin nhẹ theo mô hình Publish/Subscribe, chạy trên TCP, được thiết kế cho thiết bị IoT có tài nguyên hạn chế và mạng không ổn định. Dự án dùng Mosquitto làm broker cả trong dev lẫn production. Cấu hình local nằm ở `mqtt/docker/`, bridge EC2 ở `mqtt/config/`.

So với HTTP, MQTT có header tối thiểu 2 bytes thay vì hàng trăm bytes, duy trì kết nối TCP liên tục không cần handshake mỗi lần gửi, hỗ trợ giao tiếp hai chiều cho phép server chủ động đẩy dữ liệu xuống client, và có cơ chế QoS cùng LWT để xử lý mất kết nối.



Hình 1.7: Kiến trúc Publish/Subscribe của MQTT

1.4.2 Thành phần chính

Gateway (Z3Gateway) và cloud (FastAPI + paho-mqtt) là hai client chính. Broker cấu hình bốn principal: `gateway` có quyền readwrite trên toàn topic tree, `client` có quyền read state và write command, `monitor` có quyền read `$SYS/#` cho healthcheck, và `bridge` dùng để bridge local ↔ EC2. ACL nằm ở `mqtt/config/acl.conf`.

1.4.3 Quality of Service (QoS)

QoS xác định mức độ đảm bảo tin nhắn đến được đích.

Bảng 1.3: Các mức QoS trong MQTT

Mức	Tên	Mô tả
QoS 0	At most once	Fire-and-forget: gửi một lần, không xác nhận. Có thể mất tin nhưng nhanh nhất. Phù hợp dữ liệu telemetry liên tục.
QoS 1	At least once	Đảm bảo đến đích ít nhất một lần. Gửi lại nếu không nhận ACK. Có thể trùng lặp. Phù hợp lệnh điều khiển.
QoS 2	Exactly once	Chính xác một lần, bắt tay 4 bước (PUBLISH, PUBREC, PUBREL, PUBCOMP). Chậm nhất nhưng không mất, không trùng.

QoS được quy định trong `docs/MQTT_CONTRACT.md`. Các message command/reply quan trọng dùng QoS cao để đảm bảo delivery; telemetry định kỳ có thể dùng QoS thấp hơn. QoS áp dụng riêng cho từng chặng (Publisher→Broker và Broker→Subscriber), mức QoS thực tế là min(QoS_publish, QoS_subscribe).

1.4.4 Retained Messages và Last Will

Khi publish với cờ Retain = true, Broker lưu tin nhắn cuối cùng của topic. Subscriber mới sẽ nhận ngay tin nhắn này khi đăng ký, không cần đợi Publisher gửi lại. Một số topic trạng thái (ví dụ state gần nhất của thiết bị) dùng retain để client kết nối sau vẫn nhận được trạng thái mới nhất. Chính sách cụ thể được tra cứu trong `docs/MQTT_CONTRACT.md`.

Last Will and Testament (LWT) cho phép client khai báo một "di chúc" khi kết nối. Nếu client ngắt kết nối đột ngột, Broker tự động publish Will Message, ứng dụng phổ biến là báo trạng thái offline của gateway cho hệ thống giám sát.

1.4.5 Topics và Wildcards

Namespace MQTT của dự án có cấu trúc phân cấp:

```
sb/v1/{tenant_id}/{site_id}/{gateway_id}/...
```

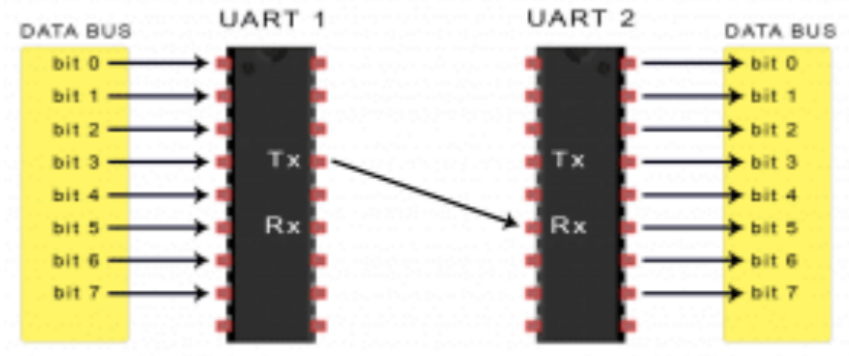
Giá trị mặc định trong môi trường phát triển là `tenant_id=hust`, `site_id=lab01`, `gateway_id=gw-ubuntu-01`. Cloud subscribe theo wildcard `sb/v1/+/+/+/...` để nhận mọi gateway.

MQTT hỗ trợ hai loại wildcard khi subscribe: dấu + khớp đúng một cấp (single-level) và dấu # khớp từ 0 đến nhiều cấp (multi-level), phải đặt cuối topic.

1.5 Giao tiếp UART

1.5.1 Tổng quan

UART (Universal Asynchronous Receiver/Transmitter) là kênh vật lý giữa host Linux (Z3Gateway) và chip EFR32MG12 chạy firmware NCP. Giao thức truyền thông nối tiếp bất đồng bộ này sử dụng hai dây TX và RX cho phép full-duplex, kết nối point-to-point, không sử dụng clock chung mà hai thiết bị phải thống nhất trước về baud rate.



Hình 1.8: Kết nối UART giữa hai thiết bị

1.5.2 Frame Format

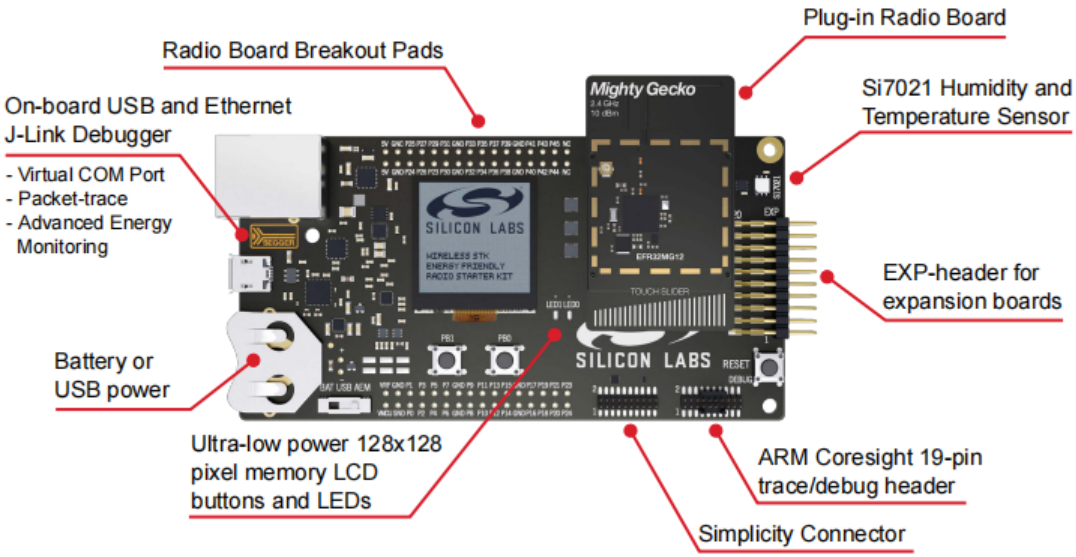
Mỗi UART frame gồm Start bit (1 bit), Data bits (thường 8 bit), Parity bit (tùy chọn) và Stop bit (1–2 bit). Khung truyền trên kênh UART trong dự án tuân thủ ASH (Asynchronous Serial Host), giao thức định dạng khung có flag byte, escape byte, CRC và flow control do Silicon Labs đặc tả và Z3Gateway xử lý sẵn.

1.5.3 Ứng dụng UART trong dự án: EZSP/ASH

Payload bên trong frame ASH là các lệnh EZSP (EmberZNet Serial Protocol): host gửi command frame (ví dụ "send ZCL On/Off") và nhận về response/callback frame (ví dụ attribute report từ end device). Dự án không tái hiện lớp framing tùy biến, EZSP/ASH được Z3Gateway xử lý nguyên bản. File `docs/UART_FRAME_FORMAT.md` chỉ còn giữ lại làm tham chiếu lịch sử cho cầu nối Python đã bị loại bỏ.

1.6 Phần cứng và Kit sử dụng

1.6.1 EFR32MG12 2.4 GHz 10 dBm Wireless Starter Kit



Hình 1.9: EFR32MG12 2.4 GHz 10 dBm Wireless Starter Kit

Kit phát triển gồm BRD4001A (Wireless Starter Kit mainboard) và BRD4162A (EFR32MG12P332F1 radio board). Chip cung cấp công suất phát 10 dBm ở 2.4 GHz và 1 MB flash đủ cho firmware Zigbee 3.0 cùng ứng dụng.

Radio Board BRD4162A là bo mạch vô tuyến dạng plug-in, sử dụng SoC EFR32MG12 Wireless Gecko hỗ trợ đa giao thức mạng lưới, với cấu hình Flash 1024 kB và RAM 256 kB. Khối thu phát vô tuyến hoạt động ở băng tần 2.4 GHz với anten tích hợp dạng Inverted-F PCB antenna. BRD4162A còn tích hợp 8 Mbit Low-power Serial Flash (MX25R) phục vụ lưu trữ dữ liệu hoặc firmware OTA.

Mainboard BRD4001A cung cấp nền tảng cấp nguồn, nạp/gỡ lỗi và ngoại vi tương tác. Mainboard tích hợp màn hình LCD Memory 128×128 pixel, cảm biến Si7021 (I2C), 2 nút nhấn (BTN0, BTN1) và 2 LED (LED0, LED1), cùng On-board Debugger (SEGGER J-Link), Advanced Energy Monitor (AEM), Packet Trace Interface (PTI) và Virtual COM Port (VCOM).

1.6.2 Cấu hình phần cứng theo thiết bị

Các firmware end device được build riêng trong thư mục `end_devices/`. Z3Light là router điều khiển đèn qua cluster On/Off 0x0006, LED0 phản ánh trạng thái đèn ON/OFF, sử dụng nguồn USB/DC. Z3Switch đóng vai trò router điều khiển/toggle, sử dụng PB0 để gửi ZCL Toggle Command và PB1 cho Network Steering. Occupancy Sensor là Sleepy End Device chạy pin, dùng PB0 để mô phỏng phát hiện hiện diện. NCP là firmware Coordinator chạy trên kit nối với gateway qua USB J-Link VCOM.

1.6.3 Toolchain và SDK

Bảng 1.4: Toolchain và SDK version

	Tên	**Version**
IDE	Simplicity Studio	5
SDK	Gecko SDK	4.5.0
Toolchain	GNU Arm	12.2.1
MCU	EFR32MG12P332F1024GL125	–
Radio Board	BRD4162A	–
Mainboard	BRD4001A	–

Build chain là Simplicity Studio cùng Gecko SDK của Silicon Labs. Gateway host dùng `Z3Gateway.Makefile` được sinh ra từ Simplicity Studio. Config cluster/radio/pin nằm trong `Z3GatewayHost/config/`, code tự sinh ở `autogen/`.

1.7 Luồng dữ liệu (Data Flow)

1.7.1 Luồng Uplink (State/Event Report)

Hình 1.10: Luồng dữ liệu Uplink

End device gửi attribute report qua ZCL tới NCP, NCP chuyển qua EZSP/ASH qua UART lên host. Tại đây `telemetry_rx.c` nhận và chuẩn hóa dữ liệu, `app_mqtt.c` publish lên topic re-

ported hoặc event tới broker Mosquitto. Cloud subscribe topic tương ứng qua `cloud/app/mqtt_client.py` mở session async SQLAlchemy và ghi vào bảng `device_states` hoặc `events`.

1.7.2 Luồng Downlink (Command)

Hình 1.11: Luồng điều khiển Downlink

Người dùng gọi `POST /api/devices/{id}/command`, module `routers/commands.py` tạo bản ghi commands với state `accepted`, `mqtt_client.py` publish lên topic command của gateway. `cmd_handler.c` parse envelope, gọi `device_dispatch.c` định tuyến theo device type, rồi `light_ctrl.c` phát ZCL frame xuống NCP. End device thực thi, trả attribute report ngược về, gateway emit reply với `correlation_id = command_id`. Cloud cập nhật `commands.state` thành `executed` hoặc `failed`.

1.7.3 Luồng Local Control

Switch bound trực tiếp với Light qua APS binding: command Zigbee đi peer-to-peer trong mạng mesh, gateway chỉ đóng vai trò lắng nghe (qua `telemetry_rx.c`) để phản ánh trạng thái mới lên cloud. Luồng này không phụ thuộc MQTT, giảm latency xuống vài chục mili-giây và chịu được mất kết nối internet.

1.8 MQTT Contract

Nguồn sự thật duy nhất cho MQTT contract là file `docs/MQTT_CONTRACT.md`.

1.8.1 Namespace và Topic Structure

MQTT namespace được thiết kế phân cấp, hỗ trợ multi-tenant và multi-site:

```
sb/v1/{tenant_id}/{site_id}/{gateway_id}/...
```

Gateway publish các kênh reported, event, reply; subscribe desired và command.

Bảng 1.5: Các nhóm MQTT topic chính

Topic Pattern	Hướng	Mô tả	QoS
`.../gateway/online`	Gw → All	Trạng thái online/offline (LWT)	1
`.../gateway/health`	Gw → Cloud	Thông tin sức khỏe gateway	0
`.../devices/.../reported`	Gw → Cloud	Trạng thái thực tế thiết bị	1
`.../devices/.../desired`	Cloud → Gw	Trạng thái mong muốn	1
`.../devices/.../event`	Gw → Cloud	Sự kiện (join, leave, error)	1
`.../command`	Cloud → Gw	Yêu cầu thực thi lệnh	1
`.../reply`	Gw → Cloud	Kết quả thực thi lệnh	1

1.8.2 Message Envelope

Mọi message MQTT đều dùng chung envelope:

Mã nguồn 1.1: MQTT message envelope

```
{
  "schema": "sb.v1",
  "msg_id": "uuid",
  "ts": "ISO8601",
  "tenant_id": "hust",
  "site_id": "lab01",
  "gateway_id": "gw-ubuntu-01",
  "source": "gateway|cloud",
  "trace_id": "optional",
  "correlation_id": "optional",
  "payload": {}
}
```

Trường `schema` cho phép consumer phân biệt và deserialize đúng kiểu message. `msg_id` là UUID duy nhất cho mỗi message, `ts` là timestamp ISO 8601, `source` cho biết nguồn phát là gateway hay cloud. `correlation_id` được set bằng `command_id` để cloud map reply về đúng lệnh gốc.

1.8.3 Retain và QoS Policy

Chính sách chi tiết tra cứu trong `docs/MQTT_CONTRACT.md`. Nguyên tắc chung: state gần nhất có thể retain cho late subscriber; command/reply không retain để tránh replay lệnh cũ. Reported state dùng QoS 1 và Retain = true, command request/reply dùng QoS 1 và Retain = false, gateway online dùng QoS 1 kết hợp Retain = true và LWT để tự động báo offline khi gateway mất kết nối.

CHƯƠNG 2
THIẾT KẾ HỆ THỐNG

Chương này trình bày quá trình thiết kế hệ thống Smart Home Platform, bao gồm phân tích yêu cầu chức năng và phi chức năng, thiết kế vai trò thiết bị trong mạng Zigbee, data model và MQTT message format, logic flow và command lifecycle, kiến trúc cloud backend, và các quyết định trade-off quan trọng.

2.1 Yêu cầu chức năng và phi chức năng

2.1.1 Yêu cầu chức năng

Hệ thống cần quản lý danh sách home, room và device qua REST API, cho phép điều khiển bật/tắt đèn thông qua cluster 0x0006 từ cloud xuống thiết bị Zigbee. Telemetry và event từ thiết bị phải được nhận và lưu bền vững vào PostgreSQL. Vòng đời lệnh được theo dõi chặt chẽ với các trạng thái chuẩn: accepted → queued → sent → executed | failed | timeout. Hệ thống cần phát hiện presence qua Occupancy Sensor, trong đó local binding có sẵn nhờ Zigbee, đồng thời cho phép join thiết bị mới qua cơ chế permit-join của Z3Gateway.

2.1.2 Yêu cầu phi chức năng

Hệ thống phải deploy được trên EC2 bằng docker-compose (`deploy/docker-compose.prod.yml`). Timeout mặc định cho command là 5000 ms (`SB_COMMAND_TIMEOUT_MS`). Broker cấu hình ACL tách biệt principal gateway/client/monitor/bridge để giới hạn quyền theo nguyên tắc least privilege. Hệ thống chịu được disconnect internet ngắn hạn nhờ local control qua binding, đảm bảo thiết bị vẫn hoạt động ngay cả khi mất kết nối cloud.

2.2 Network và Device Roles (Zigbee)

Coordinator/NCP chạy trên EFR32MG12 với firmware `end_devices/NCP`, giao tiếp host qua EZSP/ASH. Router gồm Z3Light và Z3Switch, luôn online và relay mesh. End Device là Occupancy Sensor, hoạt động ngắt quãng và gửi event khi phát hiện chuyển động.

Bảng 2.1: Ánh xạ device type và cluster chính

Device	**Zigbee Role**	**Server Clusters**	**Client Clusters**
Coordinator/NCP	ZC / Trust Center	–	On/Off Client
Light Device	ZR (Router)	On/Off (0x0006)	–
Switch Device	ZR (Router)	–	On/Off (0x0006)
Occupancy Sensor	ZED (SED)	Occupancy Sensing (0x0406)	–

Bảng ánh xạ trên tương ứng với mô tả trong `docs/DEVICE_CAPABILITY_MATRIX.md` và `docs/ADAPTER_ACTION_MAP.md`. Light sử dụng cluster On/Off 0x0006 phía server, Switch

sử dụng cluster On/Off phía client kết hợp binding, còn Occupancy Sensor sử dụng cluster Occupancy Sensing.

2.3 Data và Message Design

2.3.1 MQTT Message Channels

Hệ thống phân tách message thành năm kênh chính theo hướng truyền và ngữ nghĩa.

Bảng 2.2: Thiết kế các kênh MQTT

Channel	Hướng	Publisher	Subscriber
reported	Uplink	gateway	cloud
event	Uplink	gateway	cloud
reply	Uplink	gateway	cloud
desired	Downlink	cloud	gateway
command	Downlink	cloud	gateway

2.3.2 State Message Design

Trạng thái thiết bị publish trên kênh reported mang payload chứa `device_id` logic, eui64 phần cứng và các attribute (ví dụ `on_off: true/false` cho đèn). Cloud ghi vào bảng `device_states` trong PostgreSQL.

Mã nguồn 2.1: Light reported state message

```
{
  "schema": "sb.v1",
  "msg_id": "550e8400-e29b-41d4-a716",
  "ts": "2026-01-15T10:30:00Z",
  "source": "gateway",
  "payload": {
    "device_id": "light_01",
    "device_type": "light",
    "eui64": "00:11:22:33:44:55:66:77",
    "on_off": true,
    "endpoint": 1
  }
}
```

2.3.3 Command Message Design

Command từ cloud mang envelope chuẩn với payload chứa action cụ thể (ví dụ `{"action": "on_off", "value": true}`). `msg_id` là UUID để dedup; `correlation_id` sẽ được gateway echo lại trong reply để cloud map về `commands.id`.

Mã nguồn 2.2: Command request message

```
{
  "schema": "sb.v1",
  "msg_id": "cmd-uuid-001",
  "ts": "2026-01-15T10:31:00Z",
```

```
  "source": "cloud",
  "payload": {
    "command_id": "cmd-uuid-001",
    "target_device": "light_01",
    "action": "on_off",
    "value": true
  }
}
```

Mã nguồn 2.3: Command reply message

```
{
  "schema": "sb.v1",
  "msg_id": "reply-uuid-001",
  "ts": "2026-01-15T10:31:02Z",
  "source": "gateway",
  "correlation_id": "cmd-uuid-001",
  "payload": {
    "command_id": "cmd-uuid-001",
    "status": "executed",
    "message": "ZCL On/Off command sent successfully"
  }
}
```

2.3.4 Versioning Rule

Namespace chứa phiên bản: `sb/v1/...`; envelope có trường `schema: "sb.v1"`. Thay đổi không tương thích đòi hỏi bump lên v2, giữ song song hai phiên bản trong giai đoạn chuyển tiếp. Thêm field mới vào payload không phá vỡ consumer cũ (consumer bỏ qua field không biết), nhưng xóa field hoặc đổi kiểu dữ liệu yêu cầu nâng phiên bản.

2.4 Logic Flow và State Machine

2.4.1 Light Device State Machine

Thiết bị đèn chỉ có hai trạng thái kiên định ON và OFF đối với cluster 0x0006. Trạng thái thay đổi qua ZCL command (từ gateway hoặc switch bound trực tiếp) và phản hồi qua attribute report về `telemetry_rx.c`. Sau mỗi transition, Light Device cập nhật thuộc tính `OnOff` trong On/Off Server Cluster, đổi trạng thái LED vật lý và gửi Report Attributes lên Coordinator.

2.4.2 Command Lifecycle

Mỗi command đi qua một vòng đời với các trạng thái theo dõi rõ ràng: accepted → queued → sent → executed | failed | timeout.

Gateway phát các chuyển tiếp accepted, queued và sent. Gateway hoặc cloud timeout worker phát timeout. Device reply (qua gateway) phát executed hoặc failed. Mọi reply đều mang `correlation_id = {command_id}`, cho phép cloud cập nhật trạng thái command trong database khi nhận reply.

2.4.3 Gateway Runtime Flow

Tiến trình Z3Gateway khởi tạo theo thứ tự: `net_mgr` form network, `app_mqtt` connect broker và subscribe desired/command, `device_registry` khởi tạo bảng trống, `cmd_handler` bắt đầu loop dispatch. Khi có message command, `cmd_handler.c` giải mã envelope, kiểm tra schema, gọi `device_dispatch.c` để chọn module (`light_ctrl` hay `switch_logic`), rồi gateway emit accepted/queued/sent và chờ reply từ NCP trước khi emit executed hoặc failed.

2.5 Cloud Backend Architecture

2.5.1 Kiến trúc tổng quan

Ứng dụng FastAPI được tổ chức theo layer. `main.py` khởi tạo app cùng lifespan (database, MQTT, worker). `config.py` sử dụng pydantic-settings với prefix `SB_`. `database.py` tạo async engine và session factory. `models.py` định nghĩa bảy bảng ORM. `schemas.py` chứa Pydantic request/response. `mqtt_client.py` subscribe reported/event/reply rồi ghi vào database, đồng thời expose hàm publish cho router commands. `command_timeout.py` là worker sweep commands quá hạn sang trạng thái timeout. Thư mục `routers/` chứa các module health, devices, commands và events.

2.5.2 Database Design

PostgreSQL schema (file `database/schema.sql`, tương ứng `cloud/app/models.py`) gồm bảy bảng chính.

Bảng 2.3: Các bảng database chính

Bảng	**Vai trò**
`homes`	Đơn vị tổ chức cấp cao nhất
`rooms`	Thuộc home, nhóm thiết bị
`users`	Người dùng
`devices`	Thiết bị Zigbee (khóa logic device_id, kèm eui64)
`device_states`	Trạng thái gần nhất của từng device
`events`	Log event lâu bền
`commands`	Vòng đời lệnh với state enum

2.5.3 REST API Endpoints

Bảng 2.4: REST API endpoints chính

Method	Path	Ý nghĩa
GET	`/health`	Healthcheck
GET	`/api/devices`	Liệt kê thiết bị, filter `?room_id=`
GET	`/api/devices/{device_id}`	Chi tiết thiết bị
GET	`/api/devices/{device_id}/state`	Trạng thái gần nhất
POST	`/api/devices/{device_id}/command`	Tạo lệnh, publish MQTT
GET	`/api/commands/{command_id}`	Tra cứu lệnh theo id
GET	`/api/events`	Lọc event theo device_id, event_type, limit, offset

2.5.4 MQTT – Database Integration

`mqtt_client.py` là lớp kết nối giữa hai thế giới: subscribe các topic uplink rồi ghi database qua session async, đồng thời expose hàm publish để `routers/commands.py` gọi khi tạo lệnh mới. Worker `command_timeout.py` chạy nền, định kỳ quét bảng commands để chuyển những bản ghi quá `SB_COMMAND_TIMEOUT_MS` sang state timeout.

2.6 Trade-offs và Quyết định thiết kế

Loại bỏ cầu nối Python ↔ gateway (tháng 4/2026). Trước đây có một process Python (`gateway/src/bridge.py`) làm trung gian MQTT↔IPC qua Unix socket và NDJSON. Quyết định gộp client MQTT vào Z3Gateway C loại bỏ một process, một format wire và một lớp lỗi (latency IPC, reconnect hai phía). Đổi lại, mọi thay đổi contract MQTT phải đồng bộ trong C code thay vì Python thuần.

device_id logic thay vì eui64 hay nwk_addr. eui64 là danh tính bền nhưng gắn chặt phần cứng, nwk_addr thì đổi mỗi lần rejoin. Chọn `device_id` tự sinh giúp database ổn định qua các thay đổi hardware và tránh leak thông tin MAC.

Retain cho state, không retain cho command. State có thể an toàn retain để client mới đọc được trạng thái gần nhất; command retain sẽ gây replay sai khi gateway reconnect.

Timeout ở cả gateway lẫn cloud. Gateway phát timeout khi NCP không reply; cloud worker cũng có vòng sweep phòng trường hợp gateway không gửi được timeout (mất kết nối). Hai lớp độc lập giảm rủi ro lệnh treo vĩnh viễn.

CHƯƠNG 3
TRIỂN KHAI HỆ THỐNG

Chương này trình bày quá trình triển khai hệ thống Smart Home Platform, bao gồm firmware cho các node Zigbee (Coordinator/NCP, Light, Switch, Occupancy), Gateway Service viết bằng C với MQTT trực tiếp, Cloud Backend (FastAPI + PostgreSQL + Mosquitto) deploy trên AWS EC2, và luồng end-to-end command/state flow.

3.1 Triển khai Firmware

Firmware end device được build bằng Simplicity Studio cùng Gecko SDK. Mỗi loại thiết bị có project riêng trong thư mục `end_devices/`.

3.1.1 Cấu trúc Firmware Coordinator

Firmware `end_devices/NCP` chạy trên EFR32MG12 đóng vai trò NCP thuần, không có application logic, chỉ phơi EZSP/ASH qua UART để host Z3Gateway lái. Điều này tách biệt radio stack (trên NCP) khỏi logic ứng dụng (trên host Linux), cho phép cập nhật gateway mà không cần reflash chip radio.

3.1.2 Firmware Light Device

Firmware Z3Light là router Zigbee 3.0, implement cluster On/Off 0x0006 server side. Khi nhận ZCL command, firmware bật/tắt GPIO điều khiển đèn và gửi attribute report ngược lại để cluster client (gateway hoặc switch) biết trạng thái thực tế. LED0 trên mainboard phản ánh trạng thái đèn: ON khi sáng, OFF khi tắt.

3.1.3 Firmware Switch Device

Firmware Z3Switch đóng vai trò router, triển khai cluster On/Off client side. Sau khi bind với Light, switch phát trực tiếp lệnh Toggle tới endpoint đã bind mà không cần qua gateway, đây là cơ chế local control (Mục 1.7.3). PB0 dùng để gửi ZCL Toggle Command, PB1 dùng cho Network Steering khi join mạng.

3.1.4 Firmware Occupancy Sensor

Firmware Occupancy triển khai cluster Occupancy Sensing, phát event khi phát hiện chuyển động. Gateway lắng nghe event này qua `telemetry_rx.c` và chuyển thành message MQTT event lên cloud. Thiết bị hoạt động ở chế độ Sleepy End Device, tối ưu cho nguồn pin.

3.2 Triển khai Gateway Service

3.2.1 Thiết lập môi trường

Build bằng Simplicity Studio hoặc trực tiếp qua `Z3Gateway.Makefile`. Binary runtime cần hai thứ: UART device kết nối NCP và broker MQTT (cấu hình trong `app_config.h` hoặc biến môi trường). Không còn pytest cho gateway vì các test Python cũ thuộc cầu nối đã loại bỏ không còn giá trị.

3.2.2 Kiến trúc đa luồng

Các module C trong `Z3GatewayHost/app/` phối hợp quanh event loop chính do Z3Gateway framework cung cấp. `app_mqtt` xử lý I/O MQTT, `cmd_handler` xử lý callback message, `telemetry_rx` nhận callback ZCL attribute, `device_monitor` cập nhật presence. Tất cả chia sẻ cấu trúc `device_registry` trong RAM.

3.2.3 Giao tiếp Host–NCP: EZSP/ASH và MQTT trực tiếp

Đề cương gốc nhắc tới "UART Line Protocol" nhưng đây là kiến trúc cũ đã bị loại bỏ. Trong triển khai hiện tại, gateway không định nghĩa framing tùy biến trên UART: lớp vật lý/liên kết do ASH đảm nhiệm, lớp ứng dụng do EZSP đảm nhiệm, cả hai thuộc Silicon Labs stack. Phía cloud, gateway nói MQTT trực tiếp qua `app_mqtt.c`, không còn Unix socket `/tmp/sb-gateway.sock`, không còn NDJSON IPC, không còn process Python trung gian.

3.2.4 Cơ chế Correlation ID

Khi nhận command, `cmd_handler.c` lấy `msg_id` của envelope và echo vào `correlation_id` của mọi reply kế tiếp (accepted, queued, sent, executed/failed). Cloud dựa trên `correlation_id` để map reply về bản ghi commands gốc và cập nhật state tương ứng.

3.2.5 Chính sách Timeout và Retry

Cloud có worker `command_timeout.py` quét bảng commands, bản ghi nào quá `SB_COMMAND_TIMEOUT` (mặc định 5000 ms) mà chưa có reply executed/failed sẽ bị set state timeout. Phía gateway, nếu NCP không trả reply trong ngưỡng nội bộ, `cmd_handler.c` cũng emit timeout qua MQTT. Hai lớp timeout hoạt động độc lập nhằm giảm rủi ro lệnh treo vĩnh viễn.

3.2.6 Rules Engine

Module `rule_engine.c/h` cùng `sb_command.c/h` chứa helper cho command struct và đánh giá rule. Chi tiết rules engine hiện chỉ mới có khung module, triển khai đầy đủ vẫn đang phát triển và không có dữ liệu chi tiết hơn trong phạm vi hiện tại.

3.2.7 Admin API

Gateway không expose HTTP admin API riêng. Mọi tương tác quản trị đều đi qua MQTT (command/desired) hoặc qua REST API của cloud. Tính năng HTTP admin trực tiếp trên gateway chưa được triển khai trong phạm vi dự án.

3.3 Triển khai Cloud (FastAPI + PostgreSQL + EC2)

Đề cương gốc ghi "Firebase" là không chính xác vì dự án không sử dụng Firebase. Stack thực tế là FastAPI + PostgreSQL + Mosquitto, đóng gói Docker, deploy trên AWS EC2.

3.3.1 Thiết lập Cloud Stack

Phát triển cục bộ thực hiện qua các lệnh: `pip install -r cloud/requirements.txt` để cài đặt phụ thuộc, `python -m cloud.app.seed` để seed dữ liệu mẫu, `python -m cloud` để chạy API với Swagger tại `http://localhost:8000/docs`, và `pytest cloud/tests/` để chạy test.

Bảng 3.1: Biến môi trường Cloud (prefix `SB_`)

Biến	**Mặc định**
`SB_DATABASE_URL`	`postgresql+asyncpg://`
`SB_MQTT_HOST`, `SB_MQTT_PORT`, `SB_MQTT_USERNAME`, `SB_MQTT_PASSWORD`	–
`SB_TENANT_ID`, `SB_SITE_ID`, `SB_GATEWAY_ID`	hust, lab01, gw-ubuntu-01
`SB_API_HOST`, `SB_API_PORT`	0.0.0.0, 8000
`SB_COMMAND_TIMEOUT_MS`	5000

Deploy EC2 được thực hiện qua PowerShell: sao chép `deploy/.env.deploy.example` thành `deploy/.env.deploy`, sau đó chạy `deploy/ec2-setup.ps1` và `deploy/deploy.ps1`. Stack production (`deploy/docker-compose.prod.yml`) gồm ba container: `sb-postgres`, `sb-mosquitto` (port 1883) và `sb-cloud-api` (port 8000).

3.3.2 Device State Sync

`mqtt_client.py` subscribe topic reported dưới namespace `sb/v1/+/+/+/...`; với mỗi message hợp lệ, module mở một session async và upsert bảng `device_states` theo `device_id`. Client REST đọc lại trạng thái qua `GET /api/devices/{device_id}/state`.

3.3.3 Event History và Logging

Event (ví dụ occupancy detected, join/leave) được ghi vào bảng `events` không giới hạn thời gian. API `GET /api/events` hỗ trợ filter theo `device_id`, `event_type` và phân trang qua limit/offset. Logging hệ thống ở tầng FastAPI dùng cấu hình mặc định của Uvicorn.

3.3.4 Vấn đề thực tế khi deploy EC2

Quá trình deploy từ Windows PowerShell lên Ubuntu EC2 đã gặp và xử lý nhiều lỗi đặc thù. CRLF trong here-string PowerShell gây lỗi bash, giải pháp là strip ký tự `r` rồi SCP script trước khi thực thi. BOM UTF-8 do `Set-Content -Encoding utf8` tạo ra được xử lý bằng `UTF8Encoding $false`. `$ErrorActionPreference = "Stop"` bắt warning `mosquitto_passwd` thành lỗi, giải pháp là tạm chuyển sang Continue và parse EXIT_CODE. Directive Mosquitto lỗi thời cần cập nhật: `keepalive_interval` chỉ dùng cho bridge, `message_size_limi` đổi thành `max_packet_size`. Xung đột port 1883 giữa Mosquitto native systemd và container được xử lý bằng `systemctl stop/disable mosquitto`. Healthcheck cần credential (allow_anonymous false) nên dùng `mosquitto_sub -u monitor -P monitor123`. ACL monitor phải là `topic read $SYS/#`.

3.4 Triển khai Flutter Mobile App

Tính năng này chưa được triển khai trong phạm vi dự án. Thư mục `mobile_app/` hiện chỉ là placeholder và nằm ngoài scope v1.

3.5 Luồng End-to-end Command/State

3.5.1 Client → Cloud → Gateway → Light

Client gọi `POST /api/devices/{device_id}/command` với body mô tả action (ví dụ on_off=true). Module `routers/commands.py` validate qua Pydantic, insert bản ghi commands với state accepted, sinh `command_id` dạng UUID. `mqtt_client.py` build envelope với `msg_id = command_id`, publish lên topic command của gateway dưới namespace `sb/v1/hust/lab01/gw-ubuntu-`

`app_mqtt.c` nhận message, bàn giao cho `cmd_handler.c`. `cmd_handler.c` parse envelope, gateway emit reply accepted/queued/sent (cloud cập nhật `commands.state` tương ứng). `device_dispatch.c` định tuyến theo `device_type` trong `device_registry`, gọi `light_ctrl.c` phát ZCL command On/Off tới cluster 0x0006 của Light qua NCP. Light nhận ZCL frame, đổi state, trả attribute report. `telemetry_rx.c` bắt report, publish reported (cập nhật `device_states`) và gateway emit reply với `executed` mang `correlation_id = command_id`. Cloud cập nhật `commands.state = executed`. Nếu quá `SB_COMMAND_TIMEOUT_MS` mà chưa có executed/-failed, worker `command_timeout.py` chuyển state sang timeout.

Đường đi hoàn chỉnh đã được kiểm chứng qua commit 647b1df ("complete downstream command path cloud → gateway → Zigbee light").

3.5.2 Local Automation: Occupancy → Light

Kịch bản local automation dựa trên hai cơ chế Zigbee. Thứ nhất là Binding APS: Occupancy Sensor (hoặc một rule khởi tạo từ gateway) bound với Light qua APS, cho phép trigger On/Off không cần rời mạng Zigbee. Thứ hai là gateway lắng nghe phản ánh: `telemetry_rx.c` bắt attribute report của Light sau khi state đổi, publish reported để cloud đồng bộ, dù cloud không phải bên khởi lệnh.

Ưu điểm của luồng này là vẫn hoạt động khi mất kết nối Internet vì cả chuỗi detect → switch on nằm trọn trong mesh Zigbee. Module `rule_engine.c` dự kiến mở rộng để cho phép định nghĩa rule phức tạp hơn ngoài binding thuần.

3.6 Hướng phát triển OTA

OTA được đặc tả chi tiết trong `docs/OTA_CAMPAIGN_CONTRACT.md`. Z3Gateway xử lý artifact fetch và ZCL OTA delivery nguyên bản: host gateway tải file firmware từ nơi lưu trữ (xem `docs/FIRMWARE_ARTIFACTS.md`), sau đó phát block theo ZCL OTA cluster xuống end device. Firmware binary không bao giờ truyền qua payload MQTT; MQTT chỉ mang metadata chiến dịch (campaign id, phiên bản, target set), artifact thực tế đi qua HTTP hoặc file store.

Campaign lifecycle diễn ra như sau: cloud khởi tạo chiến dịch, gateway báo cáo tiến độ từng thiết bị, cloud tổng hợp trạng thái. Các tài liệu liên quan gồm `docs/FIRMWARE_ARTIFACTS.md` (nguồn artifact) và `docs/FLASHING.md` (quy trình nạp firmware ban đầu). OTA nằm ngoài scope commit hiện tại, được thiết kế sẵn để triển khai ở giai đoạn sau mà không phá vỡ contract MQTT `sb.v1`.

CHƯƠNG 4
KẾT QUẢ VÀ ĐÁNH GIÁ

Chương này trình bày kết quả chạy hệ thống Smart Home Platform, bao gồm điều kiện thử nghiệm, kết quả end-to-end, kết quả từng thành phần và đánh giá hiệu năng.

4.1 Điều kiện chạy thử (Test Setup)

Hardware:

- 1x Coordinator: EFR32MG12 Wireless Starter Kit (BRD4162A)
- 1x Light Device: EFR32MG12 Radio Board (BRD4162A)
- 1x Switch Device: EFR32MG12 Radio Board (BRD4162A)
- 1x Occupancy Sensor: EFR32MG12 Radio Board (BRD4162A)
- Gateway: Ubuntu Linux (laptop / Raspberry Pi)

Software:

- Simplicity Studio v5 with Gecko SDK 4.5.x
- MQTT Broker (Mosquitto / Cloud)
- Firebase (Realtime DB / Firestore)
- Flutter App (Android / iOS)

Connectivity:

- UART: 115200 baud, 8N1
- MQTT: Cloud broker, QoS 0/1
- Zigbee: Channel 11, centralized Trust Center

4.2 Kết quả End-to-end

(Viết kết quả kiểm chứng 2 luồng chính:)

- **Luồng Uplink:** Occupancy Sensor / Light → Zigbee → Coordinator → UART → Gateway → MQTT → Cloud → Flutter App
- **Luồng Downlink:** Flutter App → Cloud → MQTT → Gateway → UART → Coordinator → Zigbee → Light Device → ACK/State → App

4.3 Kết quả từng thành phần

4.3.1 Zigbee Network

(Viết kết quả: form network, join, binding, command/report hoạt động. Thêm hình khi có.)

4.3.2 Gateway Service

(Viết kết quả: UART parse, MQTT bridge, rules engine, ACK routing.)

4.3.3 Cloud Firebase

(Viết kết quả: device state sync, command relay, event logging.)

4.3.4 Flutter Mobile App

(Viết kết quả: UI screenshots, realtime update, light control.)

4.4 Đánh giá hiệu năng

(Viết đánh giá: latency end-to-end, reliability, energy profiler kết quả nếu có.)

CHƯƠNG 5
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Chương này tổng kết mức độ hoàn thành các mục tiêu đề ra, đánh giá đóng góp của đồ án, phân tích các hạn chế và đề xuất hướng phát triển tiếp theo.

5.1 Kết luận

(Viết tổng kết thành quả đạt được cho dự án smart home:)

- **Xây dựng mạng Zigbee hoạt động ổn định:** (Viết: mạng Zigbee 3.0 với 4 device types, firmware module hóa.)
- **Thiết kế giao thức truyền thông UART-MQTT:** (Viết: UART bridge, MQTT topics, CID/ACK/Retry, Rules Engine, LWT.)
- **Tích hợp Cloud Firebase:** (Viết: device state sync, command relay, event history.)
- **Phát triển Flutter Mobile App:** (Viết: light control, occupancy monitoring, realtime updates.)
- **Kiến trúc IoT end-to-end có thể tái sử dụng:** (Viết: kiến trúc từ firmware đến app, tách biệt rõ ràng, dễ mở rộng.)

5.2 Hạn chế

- **Quy mô thử nghiệm nhỏ:** Kết quả được thu thập với số node hạn chế trong phạm vi phòng. Chưa có dữ liệu về hiệu năng khi mở rộng lên nhiều node hoặc môi trường có nhiều vật cản.
- **Bảo mật chưa đầy đủ:** Hệ thống hiện tại chỉ có bảo mật mạng Zigbee. Giao tiếp MQTT có thể chưa mã hóa TLS đầy đủ. Flutter App cần bổ sung xác thực mạnh hơn.
- **Chưa có lưu trữ dữ liệu dài hạn chuyên dụng:** Event history lưu trong Firebase nhưng chưa có chính sách retention và analytics chuyên sâu.
- **Điểm lỗi đơn:** Coordinator là điểm duy nhất kết nối mạng Zigbee với Gateway. Nếu Coordinator hoặc Gateway gặp sự cố, hệ thống ngừng hoạt động.
- **Local automation hạn chế:** Chưa có rule engine phía Coordinator/Gateway cho automation phức tạp (ví dụ: occupancy → bật đèn tự động với điều kiện thời gian).

5.3 Hướng phát triển

- **Chuẩn hóa kiến trúc Gateway theo hướng Z3Gateway / NCP-host:** Phân tách rõ Coordinator (quản lý mạng Zigbee) và Gateway host (chuyển tiếp, logic, API). Migrate từ SoC Coordinator sang NCP model để Gateway có khả năng xử lý mạnh hơn.
- **Migrate Gateway sang Raspberry Pi:** Chuyển Gateway service từ Ubuntu laptop sang Raspberry Pi để tiến tới embedded gateway triển khai thực tế.
- **Tăng cường bảo mật:** MQTT TLS (port 8883), xác thực mạnh cho Firebase/App, phân quyền theo vai trò, audit log.
- **OTA (Over-The-Air firmware update):** Triển khai Gecko Bootloader + GBL file cho Zigbee OTA. Quản lý OTA campaign từ Cloud.
- **Local automation engine:** Xây dựng rule engine tại Gateway/Coordinator cho automation không phụ thuộc cloud (occupancy $\rightarrow$ light, schedule-based control).
- **Cảnh báo và thông báo:** Push notification qua Firebase Cloud Messaging khi có sự kiện: occupancy detected, device offline, gateway disconnect.
- **Mô hình nhiều nhà (multi-home):** Mỗi nhà một Gateway; Cloud Firebase quản lý tập trung nhiều home. User có thể quản lý nhiều nhà từ một app.
- **Mở rộng device types:** Thêm dimmer (Level Control Cluster), temperature sensor, door/window sensor, smart plug.

TÀI LIỆU THAM KHẢO

[1] Connectivity Standards Alliance. (2023). *Zigbee Specification*. Revision 23. Zigbee Document 05-3474-23.

[2] Zigbee Alliance. (2019). *Zigbee Cluster Library Specification*. Revision 8. Zigbee Document 07-5123.

[3] Silicon Labs. (n.d.). *UG103.2: Zigbee Fundamentals*. Deprecated version.

[4] HiveMQ. (n.d.). *MQTT Essentials: The Ultimate Guide to the MQTT Protocol for IoT Messaging*. Version 2.0.

[5] EMQ Technologies Inc. (2024). *Mastering MQTT: Your Ultimate Tutorial for MQTT*. Release: R24-3.

[6] Gryglecki, M. (HiveMQ). (n.d.). *Speaker Deck: MQTT Broker 100 - Basic*.

[7] Peña, E., & Legaspi, M. G. (2020). *UART: A Hardware Communication Protocol - Understanding Universal Asynchronous Receiver/Transmitter*. Analog Devices.

[8] Circuit Basics. (n.d.). *Basics of UART Communication*.

[9] Texas Instruments. (2010). *KeyStone Architecture Universal Asynchronous Receiver/-Transmitter (UART) User Guide*. Literature Number: SPRUGP1.

[10] Silicon Labs. (n.d.). *UG261: EFR32MG12 2.4 GHz 10 dBm Radio Board User's Guide*. (Dành cho bo mạch BRD4162A và các kit liên quan).

[11] Silicon Labs. (n.d.). *UG489: Silicon Labs Gecko Bootloader User's Guide for GSDK 4.0 and Higher*.

[12] Silicon Labs. (n.d.). *Gecko Platform Release Notes 4.4.5.0*.

[13] Silicon Labs. (n.d.). *UG100: EZSP Reference Guide*. EmberZNet Serial Protocol.

[14] Silicon Labs. (n.d.). *AN1278: Z3Gateway Example Application*. Zigbee NCP-host gateway using EZSP.

[15] Google. (n.d.). *Firebase Documentation*. https://firebase.google.com/docs.

[16] Google. (n.d.). *Firebase Realtime Database*. https://firebase.google.com/docs/database.

[17] Google. (n.d.). *Flutter Documentation*. https://docs.flutter.dev.

[18] Silicon Labs. (n.d.). *AN728: Over-the-Air Bootload Server and Client Setup*. Zigbee OTA Upgrade Cluster.